

实验二：词法分析

一、实验概述

本实验是编译原理实验系列的第二个实验。在实验一中，你已经设计了自己的编程语言并完成了语言规范文档。本实验的目标是：根据你的词法规范，实现一个完整的词法分析器（Scanner），将源代码转换为 Token 序列。

词法分析是编译器的第一个阶段，负责将字符流转换为有意义的词法单元（Token）。一个好的词法分析器不仅要正确识别合法的 Token，还要能够优雅地处理词法错误，提供有意义的错误信息。

二、实验目标

- 理解词法分析的基本原理：正则表达式、有限自动机（NFA/DFA）与 Token 识别的关系
- 掌握最长匹配原则及其在词法分析中的应用
- 能够实现一个完整的词法分析器，支持错误检测与报告
- 理解词法分析器与语法分析器之间的接口设计

三、实验内容

任务一：词法分析器实现（必做）

根据实验一中定义的词法规范，实现你的语言的词法分析器。程序读入源代码文件，输出 Token 序列（JSON 格式）。

1. 命令行接口

```
./compiler tokenize input.src
```

成功时输出（JSON 格式）：

```
{
  "tokens": [
    {"type": "KEYWORD", "value": "if", "line": 1, "col": 1},
    {"type": "ID", "value": "x", "line": 1, "col": 4},
    {"type": "OP", "value": ">", "line": 1, "col": 6},
    {"type": "NUM", "value": "0", "line": 1, "col": 8},
    {"type": "EOF", "value": "", "line": 1, "col": 9}
  ],
  "errors": []
}
```

存在词法错误时输出：

```
{
  "tokens": [ ... ],
  "errors": [
    {"type": "ILLEGAL_CHAR", "value": "$", "line": 1, "col": 6},
    {"type": "UNCLOSED_STRING", "value": "\"hello", "line": 2, "col": 1}
  ]
}
```

2. Token 输出格式要求

每个 Token 对象必须包含以下四个字段：

字段	类型	说明
type	字符串	Token 类型，必须是下表中的值之一
value	字符串	Token 的原始文本
line	整数	所在行号（从 1 开始）
col	整数	所在列号（从 1 开始）

标准 Token 类型（type 字段的合法值）：

Token 类型	说明	示例
----------	----	----

KEYWORD	语言关键字	if, while, return, int
ID	标识符	x, myVar, count
NUM	数字字面量（整数和浮点数）	42, 3.14
STR	字符串字面量	"hello", "world"
OP	运算符	+, -, *, /, ==, !=, >, <
DELIM	分隔符	(,), {, }, ;, ,
ASSIGN	赋值运算符	=, :=
EOF	文件结束标记	

注意：以上为基本类型，你可以根据自己语言的需要增加类型（如 BOOL, CHAR 等），但不能省略以上基本类型。如果你的语言不区分某类 Token，请在实验报告中说明。

3. 必做功能

(1) 基本 Token 识别

- 关键字识别：区分关键字与标识符
- 数字字面量：支持整数，如果你的语言有浮点数则需支持
- 字符串字面量：支持转义字符（至少 \n, \t, \\, \"）
- 运算符和分隔符：包括多字符运算符（如 ==, !=, <=, :=）
- 注释跳过：正确跳过单行或多行注释，不生成 Token

(2) 最长匹配原则

当多个 Token 模式都能匹配当前位置的字符时，必须选择最长的匹配。例如：

- := 应识别为一个 ASSIGN Token，而不是 : 和 = 两个 Token
- == 应识别为一个 OP Token，而不是两个 = Token
- ifx 应识别为标识符 ifx，而不是关键字 if + 标识符 x

(3) 错误检测与恢复

词法分析器必须能够检测并报告以下类型的错误：

错误类型	说明	示例
ILLEGAL_CHAR	无法识别的字符	\$, @
UNCLOSED_STRING	未闭合的字符串字面量	"hello
UNCLOSED_COMMENT	未闭合的多行注释	/* comment
INVALID_NUMBER	格式错误的数字	3. 14. 15, 1e

重要原则：遇到词法错误后，分析器应该跳过错误字符，继续分析后续内容，尽可能多地报告错误，而不是遇到第一个错误就停止。

(4) 位置信息

每个 Token 和每个错误都必须包含精确的行号和列号。行号从 1 开始计数，列号从每行第 1 个字符开始计数。Tab 符视为 1 个字符宽度。

任务二：测试用例集（必做）

编写一套测试用例，用于验证你的词法分析器的正确性，并在实验报告与演示中展示其运行结果。每个测试用例包含源代码与对应的期望输出（JSON）。

测试用例要求（≥ 10 组），需覆盖以下场景：

编号	测试场景	最少用例数
T1	关键字与标识符的区分（含关键字前缀的标识符）	2
T2	各类数字字面量（整数、浮点数、边界值）	2
T3	字符串字面量（含转义字符、空字符串）	1
T4	多字符运算符与最长匹配	2
T5	注释处理（单行、多行）	1
T6	词法错误检测（各类错误类	2

	型)	
--	----	--

任务三：必考场景验证（必做）

使用必考场景程序验证你的词法分析器（场景定义见《必考场景集（扩充版）》）：

- S1 - S5 的程序必须成功 tokenize，无词法错误
- S6（词法错误样例）的词法错误必须被正确检测并报告

在演示与报告中展示每个场景程序的 tokenize 输出结果。

任务四：正则表达式到 DFA 的可视化（选做，加分项）

实现一个工具，能够将你语言的词法规则（正则表达式）转换为 NFA，再转换为 DFA，并输出自动机的状态转换表，在报告中以图示展示。

```
./compiler visualize-dfa
```

四、交付物清单

序号	交付物	文件/格式	说明
1	词法分析器源代码	源代码文件	可使用任意编程语言
2	构建/运行说明	README.md	如何编译和运行
3	测试用例集	tests/test_XX.src + .expected.json	≥ 10 组，覆盖全部场景
4	S1 - S6 场景 tokenize 结果	scenarios/S1.tokens.json ~ S6.tokens.json	6 个 JSON 文件
5	AI 协作对话记录	conversation/ 目录 (Markdown 或导出文件)	完整对话 + 每轮 工作纪要
6	实验报告	report.pdf / report.docx	含完整功能演示流 程（图文）

五、验收方式

本实验采用过程化验收，重点考察真实的开发过程以及功能实现的一致性，而非仅看最终代码。验收由助教完成，包含以下四个环节。四个环节相互印证：对话记录、实验报告、源代码三者必须能够对应起来。

环节一：实验演示检查

- 学生现场演示（或提交完整录屏），展示编译器本阶段功能的完整运行流程，覆盖本实验全部必做功能。
- 演示需同时体现：正常输入的正确处理，以及错误输入的检测与报告。
- 助教可在演示过程中临时更换输入用例、要求你解释某段代码的作用或某个设计决策的理由。

环节二：AI 协作过程记录检查

- 提交开发全过程中与智能体（Agent IDE / AI 助手，如 Claude Code、Cursor、CodeBuddy 等）的完整对话记录。
- 记录需包含每一轮交互的三部分：（1）你向智能体提出的问题或指令；（2）智能体对该轮对话的完整响应；（3）该轮的工作纪要——你做了什么决策、为什么这样做、如何验证结果。
- 工作纪要不要求包含代码输出本身，重点是设计思路、决策理由与验证过程。
- 课程鼓励使用 AI 工具，但你必须能够独立解释每一行代码与每一个设计选择。完整、连贯的对话记录是你真正参与开发的证据。

环节三：实验报告检查

- 实验报告必须包含完整的功能演示流程（图文并茂）：对每一项功能，给出运行截图加文字说明，完整展示从输入到输出的过程。
- 报告还需涵盖：设计决策、关键算法实现思路、遇到的问题及其解决方法。

环节四：代码审核

- 助教审核源代码，要求与环节二的对话记录、环节三的报告演示流程保持一致：
 - 代码中实现的功能，应能在报告的演示流程中找到对应的展示；

- 代码的实现方式与思路，应能由对话记录与工作纪要解释其来源。
- 若出现明显不一致（例如代码中存在报告未演示、对话记录也无法解释的功能），将影响本实验成绩。

六、实验报告要求

实验报告是验收的核心材料之一，必须包含以下内容：

（1）完整功能演示流程（图文并茂）

- 逐项展示本实验的每个功能：给出输入示例、运行命令、运行截图与输出结果，并配文字说明。
- 演示流程需与源代码、AI 对话记录一致（详见第五部分验收方式）。

（2）设计与实现说明

- 你的语言有哪些词法类别？为什么这样分类？
- 最长匹配原则是如何实现的？举例说明。
- 词法错误的检测与恢复策略是什么？为什么这样设计？
- 词法分析器的整体架构，以及它如何与后续的语法分析器对接。

（3）AI 协作过程小结

- 简述你如何与智能体协作完成本实验：哪些部分由你主导设计、哪些借助 AI 实现、你如何验证 AI 产出的正确性。

七、参考资料

- Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools (Dragon Book), Chapter 3
- Appel. Modern Compiler Implementation in C/Java/ML, Chapter 2
- Grune, Jacobs. Parsing Techniques: A Practical Guide, Chapter 3
- Flex/JFlex 等词法分析器生成工具的文档（仅供参考，本实验要求手写实现）